DBMS Course Project · Project No. 06

Design and Implementation of a Database Management System for Hostel Accommodation and Student Services Management System

Mohammed Amaan Khan · 25WU0101075

Arslan Alam · 25WU0101016

Problem Description

Problem statement

- Hostel records kept in registers and spreadsheets
- Double allotment of beds, lost complaints, unclear fee dues

Objectives

- Store hostels, rooms and students in one database
- Track room allocation, service requests and fee payments
- Enforce integrity with keys and constraints
- Give staff a UI to insert, delete and view records

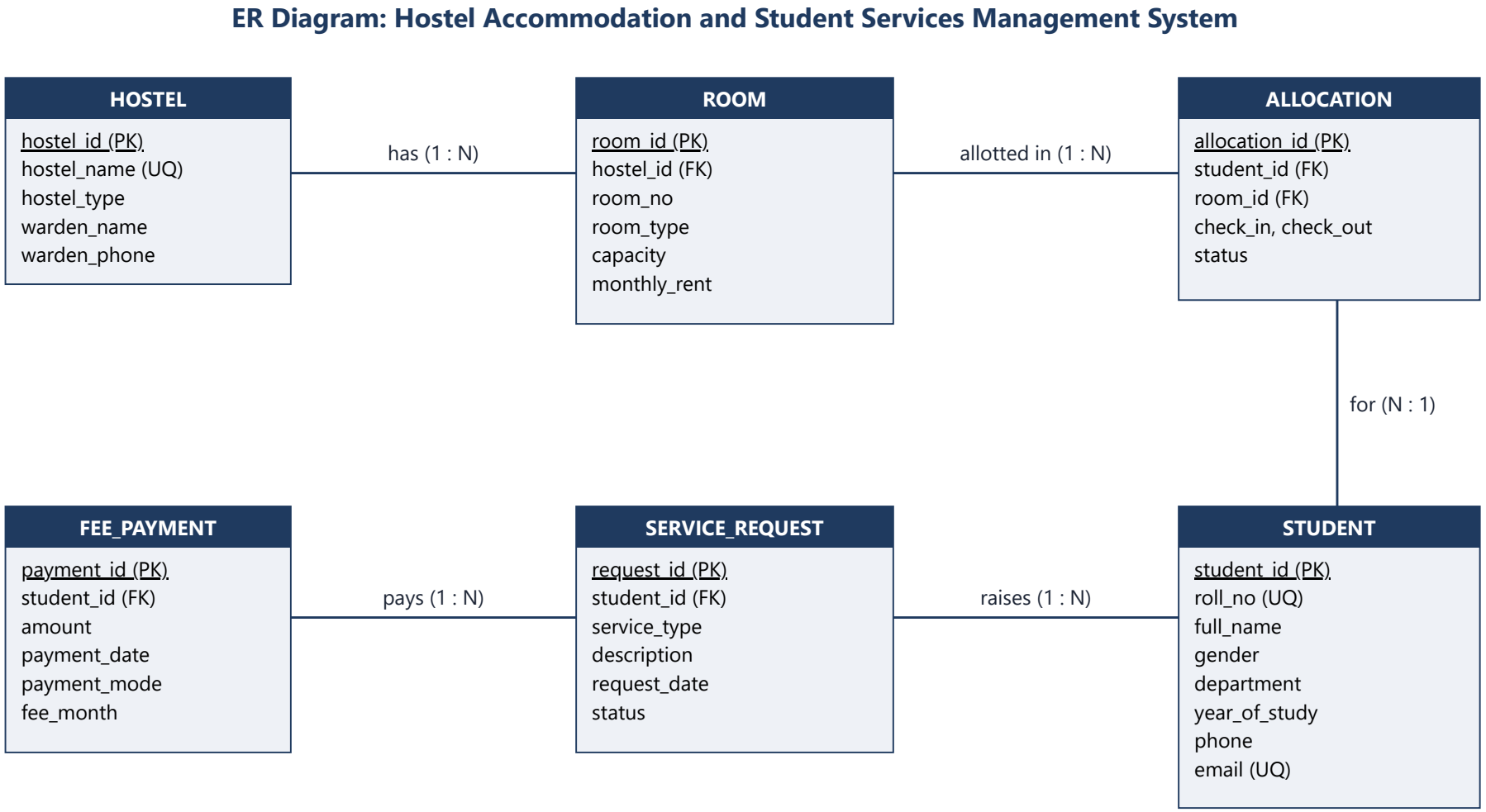
Scope

- One college, multiple hostel blocks
- Out of scope: online payment, login roles, mess menu

Intended users

- Hostel warden
- Hostel office staff

ER Diagram


ER Diagram: Hostel Accommodation and Student Services Management System
HOSTEL
hostel_id (PK)
hostel_name (UQ)
hostel_type
warden_name
warden_phone
has (1 : N)
ROOM
room_id (PK)
hostel_id (FK)
room_no
room_type
capacity
monthly_rent
allotted in (1 : N)
ALLOCATION
allocation_id (PK)
student_id (FK)
room_id (FK)
check_in, check_out
status
for (N : 1)
FEE_PAYMENT
payment_id (PK)
student_id (FK)
amount
payment_date
payment_mode
fee_month
pays (1 : N)
SERVICE_REQUEST
request_id (PK)
student_id (FK)
service_type
description
request_date
status
raises (1 : N)
STUDENT
student_id (PK)
roll_no (UQ)
full_name
gender
department
year_of_study
phone
email (UQ)

Entities and Relationships

Relationship	Cardinality	Participation
HOSTEL has ROOM	1 : N	Room total, hostel partial
ROOM allotted in ALLOCATION	1 : N	Allocation total, room partial
STUDENT gets ALLOCATION	1 : N	Allocation total, student partial
STUDENT raises SERVICE_REQUEST	1 : N	Request total, student partial
STUDENT pays FEE_PAYMENT	1 : N	Payment total, student partial

ALLOCATION resolves the M : N relationship between STUDENT and ROOM over time.

Keys: every table has a surrogate primary key; roll_no, email, hostel_name and (hostel_id, room_no) are unique.

Relational Schema and Normalization

```
hostel(hostel_id, hostel_name, hostel_type, warden_name, warden_phone)
room(room_id, hostel_id*, room_no, room_type, capacity, monthly_rent)
student(student_id, roll_no, full_name, gender, department, year_of_study, phone, email)
allocation(allocation_id, student_id*, room_id*, check_in, check_out, status)
service_request(request_id, student_id*, service_type, description, request_date, status)
fee_payment(payment_id, student_id*, amount, payment_date, payment_mode, fee_month)
```

- **1NF:** atomic values, no repeating groups
- **2NF:** single-column primary keys, so no partial dependency
- **3NF:** no transitive dependency; warden details live only in hostel, rent only in room

Database Creation and Constraints

```
CREATE TABLE room (
  room_id      INT AUTO_INCREMENT PRIMARY KEY,
  hostel_id    INT NOT NULL,
  room_no      VARCHAR(10) NOT NULL,
  room_type    ENUM('Single','Double','Triple') NOT NULL,
  capacity     INT NOT NULL CHECK (capacity BETWEEN 1 AND 3),
  monthly_rent DECIMAL(8,2) NOT NULL CHECK (monthly_rent > 0),
  UNIQUE (hostel_id, room_no),
  FOREIGN KEY (hostel_id) REFERENCES hostel(hostel_id)
);
```

- 6 tables, at least 5 records in each
- PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, ENUM
- ON DELETE CASCADE from student to allocation, service_request, fee_payment
- Live demo: run **hostel.sql** in MySQL

Queries: Joins and Aggregates

```
-- Who stays where (JOIN across 4 tables)
SELECT s.roll_no, s.full_name, h.hostel_name, r.room_no, a.status
FROM allocation a
JOIN student s ON s.student_id = a.student_id
JOIN room r    ON r.room_id = a.room_id
JOIN hostel h  ON h.hostel_id = r.hostel_id;
```

```
-- Free beds per room (LEFT JOIN, COUNT, GROUP BY)
SELECT h.hostel_name, r.room_no, r.capacity,
       r.capacity - COUNT(a.allocation_id) AS free_beds
FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
LEFT JOIN allocation a ON a.room_id = r.room_id AND a.status = 'Active'
GROUP BY r.room_id, h.hostel_name, r.room_no, r.capacity;
```

Also: total fees per student (SUM), requests per service type (HAVING), collection by month.

User Interface Demo

What the UI does

- One tab per table, plus a Reports tab
- **Insert:** form on each tab
- **View:** all rows with record count
- **Delete:** button on every row
- Constraint errors from MySQL shown on the page

Demo steps

- Insert a student, then SELECT in MySQL
- Delete a service request, then SELECT in MySQL
- Try a duplicate roll number: rejected
- Open Reports: join and aggregate results

Technology

- MySQL 8, Node.js, Express, mysql2
- HTML, CSS, JavaScript

Conclusion and Future Enhancements

Conclusion

- 3NF database covering hostels, rooms, allocation, services and fees
- Integrity enforced by the database, not by the UI
- Web UI for insert, delete and view

Future enhancements

- Login with warden and student roles
- Trigger to block allocation when a room is full
- Automatic monthly due calculation and alerts

Repository

github.com/amaancoderx/DBMS-Course-Project

Thank You

Questions?